TechnoSignia Internship — Day 13 Task

Database Fundamentals

Topics Covered

1. Introduction to Databases
2. Relational Databases
3. Tables, Rows, and Columns
4. Primary Keys and Foreign Keys
5. Relationships Between Tables
6. Introduction to PostgreSQL / MySQL
7. Database Design Basics
8. ER Diagrams

1. Introduction to Databases

1. A database is an organized collection of data that can be stored, accessed, managed, and updated easily.
2. Databases are used to store information such as student details, employee records, customer information, and product details.
3. A Database Management System (DBMS) is software used to create and manage databases.
4. A DBMS allows users to insert, retrieve, update, and delete data.
5. Examples of database management systems include MySQL, PostgreSQL, Oracle, and Microsoft SQL Server.

Example: A college database can store student names, email addresses, phone numbers, and course details.

2. Relational Databases

1. A relational database stores data in tables consisting of rows and columns.
2. Different tables can be connected using keys.
3. Each table stores information about a particular entity, such as students or courses.
4. Relationships between tables help reduce unnecessary data duplication.
5. SQL is used to communicate with relational databases.
6. MySQL and PostgreSQL are popular relational database management systems.

Example: A Student Management database can contain separate Students and Courses tables connected through an Enrollments table.

3. Tables, Rows, and Columns

1. A **Table** is a structure used to store related information in a database.
2. A **Row** represents one complete record in a table.
3. A **Column** represents a particular attribute or field of the records.
4. Each column has a name and a data type.
5. A table can contain multiple rows and columns.

Example: Students Table

student_id	student_name	email
1	Aarav	aarav@gmail.com
2	Priya	priya@gmail.com
3	Rahul	rahul@gmail.com

• Table: Students

• Row: One student's complete information.

• Column: student_id, student_name, email.

4. Primary Keys and Foreign Keys

1. A **Primary Key** uniquely identifies each record in a table.

2. A primary key cannot contain duplicate or NULL values.

3. A **Foreign Key** is a column that references a primary or unique key in another table.

4. Foreign keys establish connections between related tables.

5. They help maintain data accuracy and referential integrity.

Example:

• `student_id` in the Students table is the primary key.

• `student_id` in the Enrollments table is a foreign key referencing Students.

query.sql

```sql
-- Students table
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(100)
);

-- Enrollments table
CREATE TABLE Enrollments (
    enrollment_id INT PRIMARY KEY,
    student_id INT,
    FOREIGN KEY (student_id)
    REFERENCES Students(student_id)
);
```

5. Relationships Between Tables

1. Relationships describe how records in one table are connected to records in another table.

2. **One-to-One (1:1):** One record in a table is associated with one record in another table.

3. **One-to-Many (1:M):** One record in a table can be associated with multiple records in another table.

4. **Many-to-Many (M:N):** Multiple records in one table can be associated with multiple records in another table.

5. Relationships are implemented using primary keys and foreign keys.

6. A junction table is commonly used to implement a many-to-many relationship.

Examples:

• One-to-One: One student has one student profile.

• One-to-Many: One department has many employees.

• Many-to-Many: Many students can enroll in many courses.

6. Introduction to PostgreSQL / MySQL

1. MySQL and PostgreSQL are relational database management systems.

2. Both are used to create databases, store records, and execute SQL queries.

3. MySQL is widely used in web applications and backend development.

4. PostgreSQL is an advanced open-source relational database known for its extensibility and support for complex queries.

5. Both support SQL commands such as CREATE, INSERT, UPDATE, DELETE, and SELECT.

6. MySQL Workbench is a graphical tool used to manage MySQL databases.

7. pgAdmin is a graphical tool commonly used to manage PostgreSQL databases.

Examples:

• MySQL: `CREATE DATABASE student_management;`

• PostgreSQL: `CREATE DATABASE student_management;`

7. Database Design Basics

1. Database design is the process of planning how data will be stored and organized.

2. It involves identifying entities, attributes, relationships, and constraints.

3. An **Entity** represents a real-world object, such as a student or course.

4. An **Attribute** describes the properties of an entity, such as student name, email, or age.

5. A **Primary Key** uniquely identifies each record.

6. A **Foreign Key** connects related tables.

7. Normalization helps reduce duplicate data and improve database organization.

8. A good database design improves data consistency, accuracy, and maintenance.

Example: A Student Management database can contain:

• Students — Student information.

• Courses — Course information.

• Enrollments — Student-course relationships.

8. ER Diagrams

1. ER stands for Entity-Relationship.

2. An ER diagram is a visual representation of entities, their attributes, and relationships.

3. Entities are represented using rectangles.

4. Attributes describe the properties of entities.

5. Primary keys are commonly marked with PK.

6. Foreign keys are commonly marked with FK.

7. Relationships show how entities are connected.

8. ER diagrams help developers understand and plan database structures before implementation.

Common ER diagram symbols:

Symbol	Meaning
Rectangle	Entity
Oval	Attribute
Diamond	Relationship
Underlined attribute	Primary Key
Connecting line	Relationship connection

Practical Tasks and Solutions

Task 1 — Install or Access MySQL

query.sql

```sql
-- Display the installed MySQL server version
SELECT VERSION();
```

If the version appears in the output, MySQL is accessible and ready to use.

Task 2 — Create a Student Management Database Design

The Student Management database will contain three main tables:

1. **Students:** Stores student information.
2. **Courses:** Stores course information.
3. **Enrollments:** Connects students with their selected courses.

Database Name: `student_management`

Table 1 — Students

Field Name	Data Type	Constraint	Description
student_id	INT	Primary Key, Auto Increment	Unique student ID
student_name	VARCHAR(100)	NOT NULL	Student name
email	VARCHAR(100)	UNIQUE, NOT NULL	Student email
phone	VARCHAR(15)	—	Contact number
age	INT	CHECK (age >= 18)	Student age

Table 2 — Courses

Field Name	Data Type	Constraint	Description
course_id	INT	Primary Key, Auto Increment	Unique course ID
course_name	VARCHAR(100)	NOT NULL	Course name
duration_months	INT	—	Course duration
fees	DECIMAL(10,2)	—	Course fees

Table 3 — Enrollments

Field Name	Data Type	Constraint	Description
enrollment_id	INT	Primary Key, Auto Increment	Unique enrollment ID
student_id	INT	Foreign Key	References Students
course_id	INT	Foreign Key	References Courses

Task 3 — Identify Fields Required for Students and Courses

Students Entity:

1. `student_id` — Uniquely identifies each student.
2. `student_name` — Stores the student's full name.
3. `email` — Stores the student's email address.
4. `phone` — Stores the student's contact number.
5. `age` — Stores the student's age.

Courses Entity:

1. `course_id` — Uniquely identifies each course.
2. `course_name` — Stores the course name.
3. `duration_months` — Stores the course duration.
4. `fees` — Stores the course fees.

Enrollments Entity:

1. `enrollment_id` — Uniquely identifies each enrollment.
2. `student_id` — Identifies the enrolled student.
3. `course_id` — Identifies the selected course.

Task 4 — Understand the Relationships Between Tables

1. One student can enroll in multiple courses.
2. One course can have multiple students.
3. Therefore, Students and Courses have a many-to-many relationship.
4. The Enrollments table acts as a junction table.
5. The `student_id` foreign key connects Enrollments with Students.
6. The `course_id` foreign key connects Enrollments with Courses.

Relationship Representation:

Relationship Representation

```
Students (1) ——— (Many) Enrollments (Many) ——— (1) Courses
```

This means:

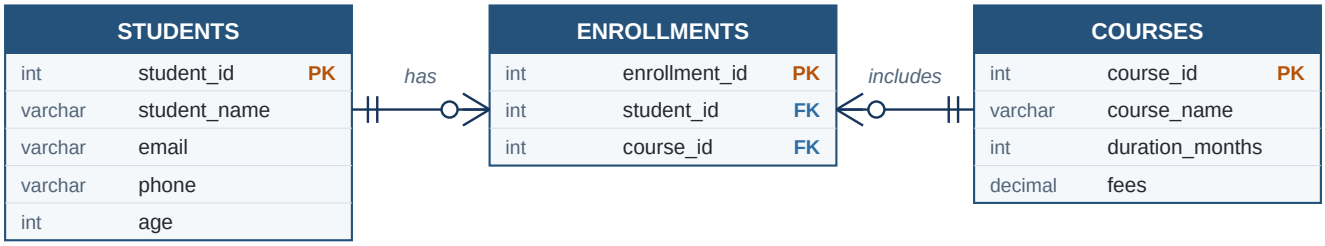
• One student can have multiple enrollment records.

• One course can appear in multiple enrollment records.

• Each enrollment belongs to one student and one course.

Task 5 — Draw the ER Diagram

The following ER diagram represents the Student Management database.



Explanation of the ER Diagram:

1. `STUDENTS`, `COURSES`, and `ENROLLMENTS` are the three entities.
2. `student_id` is the primary key of Students.
3. `course_id` is the primary key of Courses.
4. `enrollment_id` is the primary key of Enrollments.
5. `student_id` in Enrollments is a foreign key referencing Students.
6. `course_id` in Enrollments is a foreign key referencing Courses.
7. The diagram shows a one-to-many relationship from Students to Enrollments.
8. It also shows a one-to-many relationship from Courses to Enrollments.

Task 6 — Create the Database and Tables in MySQL

Step 1: Create Database

query.sql

```sql
-- Create the Student Management database
CREATE DATABASE student_management;

-- Select the database
USE student_management;
```

Step 2: Create Students Table

query.sql

```sql
-- Create the Students table
CREATE TABLE Students (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    student_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(15),
    age INT CHECK (age >= 18)
);
```

Step 3: Create Courses Table

query.sql

```sql
-- Create the Courses table
CREATE TABLE Courses (
    course_id INT PRIMARY KEY AUTO_INCREMENT,
    course_name VARCHAR(100) NOT NULL,
    duration_months INT,
    fees DECIMAL(10, 2)
);
```

Step 4: Create Enrollments Table

query.sql

```sql
-- Create the Enrollments table
CREATE TABLE Enrollments (
    enrollment_id INT PRIMARY KEY AUTO_INCREMENT,
    student_id INT NOT NULL,
    course_id INT NOT NULL,

    FOREIGN KEY (student_id)
    REFERENCES Students(student_id),

    FOREIGN KEY (course_id)
    REFERENCES Courses(course_id)
);
```

Step 5: Verify Database and Tables

query.sql

```sql
-- Display all databases
SHOW DATABASES;

-- Select the Student Management database
USE student_management;

-- Display all tables
SHOW TABLES;

-- Display Students table structure
DESCRIBE Students;

-- Display Courses table structure
DESCRIBE Courses;

-- Display Enrollments table structure
DESCRIBE Enrollments;
```

Task 7 — Add Sample Records to Understand the Design

Insert Students

query.sql

```sql
-- Add sample student records
INSERT INTO Students (student_name, email, phone, age)
VALUES
('Aarav Sharma', 'aarav@gmail.com', '9876543210', 21),
('Priya Patil', 'priya@gmail.com', '9876543211', 22),
('Rahul Deshmukh', 'rahul@gmail.com', '9876543212', 20);
```

Insert Courses

```sql
-- Add sample course records
INSERT INTO Courses (course_name, duration_months, fees)
VALUES
('Java', 6, 15000.00),
('Python', 4, 12000.00),
('React.js', 3, 10000.00);
```

Insert Enrollments

```sql
-- Connect students with their selected courses
INSERT INTO Enrollments (student_id, course_id)
VALUES
(1, 1),
(1, 3),
(2, 2),
(3, 1);
```

Display the Records

```sql
-- Display all students
SELECT * FROM Students;

-- Display all courses
SELECT * FROM Courses;

-- Display all enrollments
SELECT * FROM Enrollments;
```

Task 8 — Verify the Relationships Using JOIN

```sql
-- Display student names and their enrolled courses
SELECT
    Students.student_name,
    Courses.course_name
FROM Enrollments
JOIN Students
    ON Enrollments.student_id = Students.student_id
JOIN Courses
    ON Enrollments.course_id = Courses.course_id;
```

Explanation:

1. JOIN combines records from multiple related tables.
2. Students are connected using `student_id`.
3. Courses are connected using `course_id`.
4. The result displays each student's name along with the course they selected.